

# AcquirerX — End-to-End Story Guide

A plain-English walkthrough of what your system actually does. No jargon left undefined. Follow the story of one fictional coffee shop from "I want to accept cards" all the way to "the bank paid me, and one customer disputed a charge."

If you ever feel lost in the codebase, come back here and re-read the relevant chapter. The code is just an implementation of these business steps.

## How to read this guide

- **Bold terms in «guillemets»** are the technical words you'll see in the code. The first time each appears, I define it right there.
- 🏪 = Priya's Coffee Shop (our example merchant)
- 🏦 = AcquirerBank (the bank running AcquirerX — the platform your system models)
- 💳 = Customer with a Visa card
- 🏛️ = Visa/Mastercard (the «card network»)
- 👤 = Bank staff using AcquirerX

## The cast

Before the story starts, here's who's involved:

| Role                 | Who they are in real life                                                    | Role name in your code                                       |
|----------------------|------------------------------------------------------------------------------|--------------------------------------------------------------|
| 🏦 The acquiring bank | "AcquirerBank" — the bank that signs up merchants and lets them accept cards | (the company that owns AcquirerX)                            |
| 🏪 The merchant       | "Priya's Coffee Shop"                                                        | MERCHANT_OPS (when bank staff onboards them)                 |
| 💳 The cardholder     | "Rahul" who walks in to buy a coffee                                         | (not a user of AcquirerX directly)                           |
| 🏛️ The card network  | Visa, Mastercard, RuPay                                                      | "Network" — V, M, U, LocalSim                                |
| 🏦 The issuer         | "Rahul's bank" — HDFC, ICICI, etc. — the bank that gave him the card         | (lives outside AcquirerX)                                    |
| 👤 Bank staff         | The people who use AcquirerX day-to-day                                      | 6 roles: ADMIN, MERCHANT_OPS, POS_OPS, RISK, DISPUTES, RECON |

## Mini-glossary — read this first

Don't worry about memorizing. Skim once, come back as needed.

### People & businesses

- **«Merchant»** — a business that wants to accept cards. Priya's Coffee Shop is a merchant.

- «**Acquirer**» — the bank that *signs up* the merchant. AcquirerBank in our story. They give the merchant the POS machine and pay them the daily settlements.
- «**Issuer**» — the bank that issued the card to the customer (e.g. HDFC issued Rahul's Visa card).
- «**Cardholder**» — the person who owns the card. Rahul.
- «**Card network**» / «**Scheme**» — Visa, Mastercard, RuPay, AmEx. The middlemen who route transactions from the acquirer to the issuer. Sometimes called "Scheme" (e.g. "scheme fee").

## Things on the shop counter

- «**POS**» — Point of Sale. The cash counter where customers pay. *Not* a software term, just "the till".
- «**Terminal**» / «**POS terminal**» — the actual card-reading device on the counter. The thing where the customer taps/inserts/swipes. Brand examples: Verifone, Ingenico, PAX.
- «**TID**» — Terminal ID. A unique number the bank assigns to each terminal device. 8 digits in our system. Think of it as the "license plate" for the card reader.
- «**MID**» — Merchant ID. A unique number the bank assigns to each merchant. The "license plate" for the business.

## How cards work

- «**PAN**» — Primary Account Number. The 16-digit number printed on the front of the card. Sensitive — banks aren't allowed to store the full number after the transaction.
- «**Masked PAN**» — the safe-to-store version. First 6 digits + asterisks + last 4 digits. Example:  
453201\*\*\*\*\*0366 .
- «**BIN**» — Bank Identification Number. The first 6 digits of a PAN. They identify which bank issued the card.  
453201 = an HDFC Visa card, for example.
- «**EMV**» — the global standard for **chip cards** (the ones you insert into the terminal). Named after the three companies that defined it: Europay, Mastercard, Visa. When you "dip" a card, EMV is what's running.
- «**Contactless**» / «**CTLS**» / «**NFC**» — tap-to-pay. You wave the card or phone near the reader; no PIN needed below a small amount (₹5,000 in India by default).
- «**Magstripe**» — the old black stripe on the back of the card. Swipe-style. Still supported, much less secure.
- «**CNP**» — Card-Not-Present. Online purchases — the card was never physically at a terminal. (Phase-1 of AcquirerX focuses on card-present POS, not e-commerce.)

## Money words

- «**Authorization**» / «**Auth**» — the bank asking the issuer "is this card good for ₹X right now?" before charging. The auth either gets **APPROVED** (with an Auth Code like 045123 ) or **DECLINED** (with a Response Code).
- «**Settle**» / «**Settlement**» — the once-a-day process of bundling all the day's approved transactions and actually moving the money. Auth says "we *will* take ₹500"; settlement says "we *did* take ₹500."
- «**Batch**» — a collection of approved transactions on one terminal that haven't been settled yet. End of day, the batch is "closed" and goes to settlement.
- «**Payout**» — the actual bank transfer from AcquirerBank to the merchant's bank account. The merchant's view of "I got paid".
- «**MDR**» — Merchant Discount Rate. The total percentage the merchant pays the bank for the privilege of accepting cards. Typical Indian retail MDR ≈ 2%. So on a ₹1,000 sale, the merchant gets ₹980 and the bank/network keep ₹20.
- «**Interchange**» — the fee the *acquirer* pays the *issuer*. Of the ₹20 above, maybe ₹15 is interchange (going to Rahul's bank).
- «**Scheme fee**» — fee Visa/Mastercard charge for using their network. Maybe ₹1 of that ₹20.
- «**Acquirer markup**» — what AcquirerBank keeps as profit. Maybe ₹4 of the ₹20.
  - So: **MDR = Interchange + Scheme + Markup**

- «**IC++**» / «**IC plus plus**» — a pricing model where the merchant sees those three components separately on their statement instead of one bundled MDR. More transparent.
- «**Adjustment**» — a manual debit or credit on a merchant's account. Used when something needs correcting — e.g. customer disputed a charge, fee was calculated wrong, etc.

## Behind-the-scenes ops

- «**Onboarding**» — the entire process of setting up a new merchant: KYC, pricing, settlement profile, stores, terminals.
- «**KYC**» — Know Your Customer. The legal requirement to verify a merchant is a real, legit business. PAN card, GST, bank proof, incorporation certificate, etc.
- «**Provision**» / «**Provisioning**» — *deploying* something so it's ready to use. "Provisioning a terminal" = configuring a TID for the device, attaching parameter profile, marking it Active. Just a fancy word for "set up".
- «**Parameter profile**» — a bundle of settings for a terminal (chip kernel version, contactless tap limit, receipt copies, idle timeout). Multiple terminals can share one profile.
- «**Reconciliation**» / «**Recon**» — proving the books match. AcquirerX has its own list of approved transactions. The bank that holds AcquirerBank's funds also has a list. Visa has a list. End of day we compare the three and flag anything that doesn't match.
- «**Exception**» — a recon mismatch. e.g. AcquirerX thinks 100 transactions happened but the bank's file shows only 99 — that 1 missing one is an Exception. An analyst investigates and either resolves it or "writes it off".
- «**Settle**» (vs «**Settlement**») — same word in two meanings:
  1. The verb *to settle* a transaction = mark it as paid out.
  2. The noun *settlement* (capital S) = the daily process / the resulting batch record.
- «**Chargeback**» — the customer phones their bank and says "I never bought this, give me my money back." The issuer pulls the money back from AcquirerBank, who pulls it from Priya. That whole tug-of-war is a chargeback.
- «**Dispute**» — the umbrella term for any "I'm contesting this charge" workflow. Goes through 4 stages:
  1. «**Retrieval**» — issuer asks "give me a copy of the receipt"
  2. «**Chargeback**» — issuer pulls the money back
  3. «**Representment**» — merchant fights back: "no, here's proof he bought it"
  4. «**Arbitration**» — neither side gives up; Visa/Mastercard make the final call
- «**Risk**» — the system that decides whether a transaction is fraudulent before approving it. Rules like "decline if amount > ₹50,000 from a new card" or "block this PAN we know is stolen".
- «**Blacklist**» — a list of PANs / Terminals / Merchants that should be auto-blocked.

## Tech words

- «**Microservice**» — your backend isn't one big program; it's split into separate small servers. Each handles one area (auth, merchant, terminal, transaction, etc.). They talk to each other over HTTP.
- «**API Gateway**» — single entry point for the frontend. The browser only knows port 9090 (the gateway). The gateway figures out which microservice should handle each URL and forwards it.
- «**Eureka**» — the service registry. Microservices "check in" to Eureka at boot saying "I'm here, port 9091". The gateway asks Eureka "where's MERCHANT-SERVICE right now?" before forwarding a request. Like a phone book for services.
- «**Feign**» — when one microservice needs to call another (e.g. `terminal-service` needs to ask `merchant-service` "what merchant owns this store?"), it uses Feign — a Java client that hides the HTTP details and looks like a normal method call.
- «**JWT**» — JSON Web Token. The "ticket" you get after logging in. Every other API request must include this token in a header. The server checks the ticket is valid before serving the request.

- «**RBAC**» — Role-Based Access Control. The thing that says "this URL is only for ADMINS". Implemented in the sidebar (which links you see) and on every backend endpoint (which API calls you can make).

OK. Now the story.

---

## Chapter 1 — The bank decides to use AcquirerX

🏦 AcquirerBank is a real bank. They want to start onboarding shops to accept Visa/Mastercard. Building a payment platform from scratch is hard, so they buy AcquirerX (your software) to run that operation.

The bank's IT team installs AcquirerX:

- The 9 backend microservices boot up and «**register**» with Eureka
- The MySQL database has 7 schemas (one per microservice)
- The frontend goes live at the bank's internal URL (in development: <http://localhost:5173>)

Now they need user accounts for their staff.

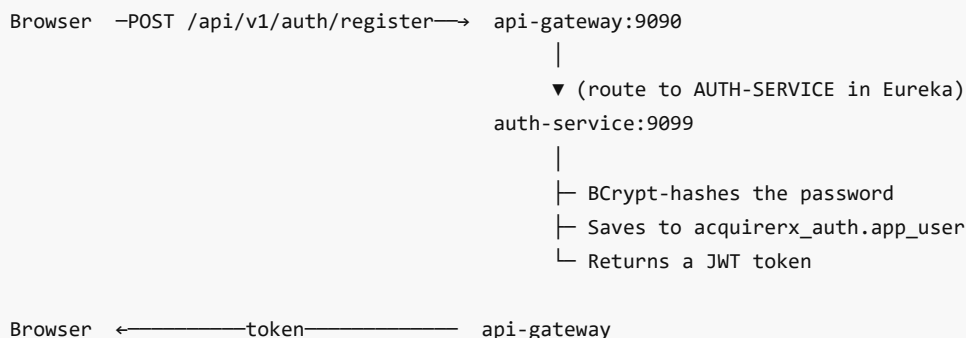
---

## Chapter 2 — Day 0: Creating the first admin

The bank's IT lead, **Karthik**, opens AcquirerX for the first time.

1. He goes to <http://localhost:5173> → automatic redirect to `/login`
2. He clicks **Register** at the bottom
3. He fills in the form:
  - Full Name: `Karthik IT Lead`
  - Phone: `+91 9876500001`
  - Username: `karthik`
  - Email: `karthik@acquirerbank.com`
  - Password: `KarthikSecure@2026`
  - Role: **Admin**
4. Clicks **Register**

**What happens behind the scenes:**



The password is **never** stored in plain text. Even Karthik's database admin can't read it. BCrypt is a one-way hashing algorithm — given the hash you can't get back the password. At login, the system hashes what you typed and compares hashes.

The JWT token Karthik now holds looks like this when decoded:

```
{
  "sub": "karthik",
  "role": "ADMIN",
  "userId": 1,
  "exp": 1714999999
}
```

The browser stashes this in «**Local Storage**» under the key `ax_token` . Every API call going forward sends `Authorization: Bearer <token>` so the server knows who's calling.

🚨 **Security thought:** *Anyone with that token is Karthik for 24 hours (token expiry). That's why we never paste tokens into Slack or screenshot them.*

Karthik creates 5 more colleagues — one per role. Now there's:

| Username | Role         | Real-world job                                     |
|----------|--------------|----------------------------------------------------|
| karthik  | ADMIN        | IT lead — does everything                          |
| meera    | MERCHANT_OPS | Onboarding officer — signs up new shops            |
| prasad   | POS_OPS      | Field engineer — installs and maintains terminals  |
| raj      | RISK         | Fraud analyst — watches for suspicious patterns    |
| divya    | DISPUTES     | Disputes officer — handles chargebacks             |
| ravi     | RECON        | Reconciliation officer — daily settlements & books |

## Chapter 3 — A new merchant signs up: Priya's Coffee Shop

🇮🇳 Priya runs a small coffee shop in Bengaluru. Today she walks into AcquirerBank's branch and says "I want to accept cards."

**Meera** (Merchant Ops) sits at her desk and starts the onboarding.

### Step 3.1 — Create the merchant record

Meera logs in ( `meera` / `password` ), goes to **Merchants** → **+ New Merchant**, and fills:

- **Legal Name:** `Priya's Coffee Shop Pvt Ltd`
- **DBA:** `Priya's Café` (DBA = "Doing Business As" — the public-facing name on the signage. Could be different from the legal name on paper.)
- **MCC:** `5812` (MCC 5812 = "Eating Places, Restaurants" in the Visa MCC list. Visa cares about MCC because it determines interchange rates — restaurants have different fees from grocery from gas stations.)
- **Contact Info:** `priya@coffee.example.com`
- **Risk Level:** `LOW` (Meera can override this. Bank policy: small cafés are LOW risk; jewellery shops are HIGH; gambling is CRITICAL.)

Click **Create Merchant**.

**What you see:** the merchant detail page opens. Big yellow badge: `Pending` .

**Why Pending?** Because Priya hasn't proved she's a real business yet. Bank policy (and Indian law) says: no card processing until KYC is on file.

🔍 In the database, a new row in `acquirerx_merchant.merchant` :

```
merchant_id: 4
legal_name:  "Priya's Coffee Shop Pvt Ltd"
status:      "PENDING"
mcc:         "5812"
risk_level:  "LOW"
```

### Step 3.2 — KYC (Know Your Customer)

Priya hands Meera a folder with her documents. Meera goes to the **KYC** tab on the merchant page:

- Click + **Submit KYC**
- Document Type: `PAN_CARD` (In India, PAN card = Permanent Account Number, a tax ID. Note: this PAN is different from card-PAN. Same word, two meanings.)
- Document Reference: `AAACT2727Q` (the actual PAN number from the card)
- Notes: `Verified original copy`
- Submit

**Watch the merchant status badge.** It just flipped from `Pending` → `Active` 🟢

That's a business rule we built in: the moment the **first** KYC document is submitted, the merchant becomes Active. (In a real bank, you'd want a "verified" step before going live. We've simplified to "submitted = trusted enough" for the demo.)

🔍 Now there are two rows: the merchant is `ACTIVE` , and a new row in `merchant_kyc` with `status='PENDING'` .

Meera also submits:

- `GST_CERT` — `27AAACT2727Q1ZT` (Indian GST registration)
- `BANK_PROOF` — `HDFC0001234-cancelled-cheque` (bank account proof)

Each one becomes another row in `merchant_kyc` . Meera will later mark them all `VERIFIED` once a senior officer eyeballs them.

### Step 3.3 — Pricing model

Now she sets the fee structure. Click **Pricing** tab → + **Add Pricing Model**.

- Model Type: `MDR` (simplest model — Priya pays a flat percentage; the bank handles the breakdown internally. The other options are `IC_PLUS_PLUS` for fee-transparent merchants who want to see interchange and scheme separately, and `BLENDED` for averaged-out fees.)
- MDR %: `2.0` (Priya pays 2% on every transaction)
- Per-Txn Fee: `2.50` (plus ₹2.50 fixed per transaction — common in India)
- Scheme Fee Pass-Through: `No` (if Yes, Visa's scheme fee gets billed to Priya separately. We're keeping it inside the 2% to make it simple.)
- Effective From: today
- Effective To: leave blank (open-ended; valid forever until deactivated)

Click **Create**. Now Priya has an `Active` pricing row.

**i Why pricing models can never be reactivated:** Imagine Priya processed a ₹1,000 sale on Tuesday, fee = 2%. On Wednesday Meera deactivates that pricing and adds a new 2.5% one. Should Tuesday's transaction retroactively pay 2.5%? **No** — that would be unfair and audit-hostile. So old pricing rows stay forever as immutable history. The frontend has an info banner explaining this.

### Step 3.4 — Settlement profile

When does Priya get paid? Click **Settlement** tab → + **Add Profile**:

- Settlement Cycle: `T_PLUS_1` (money lands one day after the transaction.  $T$  = transaction day.  $T+0$  means same day;  $T+1$  next day;  $T+2$  two days later. Banks usually offer  $T+1$  for a small extra fee — Priya wants quick cashflow.)
- Bank Account Ref: `HDFC0001234-00998877` (her business account)
- Reserve %: `5` (the bank holds back 5% of every settlement for ~6 months as protection against future chargebacks. So on a ₹1,000 sale, ₹950 goes out today, ₹50 stays in a reserve account. Released later if no disputes happen.)

Click **Create**.

### Step 3.5 — Store

A merchant can have many stores. Click **Stores** tab → + **Add Store**:

- Store Name: `Indiranagar Branch`
- Region: `APAC`
- Address: `100 Feet Road, Indiranagar`
- City: `Bengaluru` · State: `KA` · Pincode: `560038`
- Contact Person: `Priya Singh`
- Contact Phone: `+91 9876500100`

Click **Create Store**. Database row in `store` table; `merchant_id` foreign key links it to Priya's merchant record.

**Priya's onboarding is done.** What we have so far:

```
Merchant (Priya's Coffee Shop, ACTIVE, MCC 5812, Risk LOW)
├─ KYC docs: PAN_CARD ✓, GST_CERT ✓, BANK_PROOF ✓
├─ Pricing: MDR 2.0% + ₹2.50 per txn
├─ Settlement: T+1 to HDFC bank, 5% reserve
└─ Store: Indiranagar Branch (Bengaluru)
```

But she can't accept cards yet — there's no terminal at the counter.

---

## Chapter 4 — The terminal arrives

 The next day, Prasad (POS\_OPS field engineer) drives over with a Verifone V200c terminal in a box. He plugs it in at Priya's counter, then opens AcquirerX on his tablet.

### Step 4.1 — Provision the terminal

Logged in as `prasad`. **Provisioning** = "set this device up so it can accept cards". Click **Terminals** → + **New Terminal**:

- **Merchant:** pick `Priya's Coffee Shop Pvt Ltd`

- **Store:** pick `Indiranagar Branch` (only stores under the chosen merchant appear in the dropdown — a chained filter)
- **TID:** `12345678` (printed on a sticker on the bottom of the device. Prasad types the same number into the system and onto a label he sticks on top of the device. From now on, anything that happens at this terminal is tagged with TID 12345678.)
- **Brand/Model:** `Verifone V200c`
- **Capability:** `EMV` (the device can accept chip-card insertions. We could also pick `CTLS` for tap-to-pay or `Magstripe` for swipe-only.)

Click **Create**.

#### Behind the scenes:



The Feign call between services is what makes the microservice architecture useful — terminal-service knows about terminals, merchant-service knows about merchants and stores. Neither owns the other's data; they ask politely over HTTP.

## Step 4.2 — Set parameter profile

Different shops want different terminal behaviour. A coffee shop wants 1 receipt copy and a 30-second idle timeout. A jewellery shop wants 2 receipts, longer timeout, and a higher contactless limit.

Prasad creates a **parameter profile** (or reuses an existing one). On the terminal's **Profile** tab → **+ New Profile**:

- Profile Name: `STD_CAFE_v1`
- EMV Kernel: `v1.4` (EMV kernel = the chip-reading firmware version. Higher version = better fraud detection. Standard nowadays.)
- Contactless Limit: `5000` (the maximum tap-amount without PIN. Above ₹5,000, even tap requires PIN entry. Indian regulator's rule.)
- Receipt Copies: `1` (Priya prints 1 receipt for the customer. No duplicate for her records — it's all in AcquirerX.)
- Idle Timeout: `30`
- Auto-Batch Close (hr): `23` (at 11 PM the terminal auto-closes its day's batch — see Chapter 6.)
- Currency: `INR`

Click **Create Profile**. Then in the dropdown, pick the new profile and click **Assign**. The terminal now has these settings.



Behind the scenes the profile is stored as a JSON blob:

```
{
  "emvKernel": "v1.4",
  "ctlsLimit": 5000,
  "receiptCopies": 1,
  "idleTimeoutSec": 30,
  "autoBatchCloseHour": 23,
  "currency": "INR"
}
```

In a real-world system, the next time the terminal phones home it pulls these settings down. Our system simulates that — it just records the assignment in the DB.

### Step 4.3 — Open the day's batch

Before any sale can happen, Prasad opens the day's batch. Terminal detail page → **Batch Control** card → click **Open Batch**.

 New row in `acquirerx_transaction.batch` :

```
batch_id:      1
terminal_id:   1
open_time:     2026-05-08 09:00:00
close_time:    null
status:        OPEN
```

Why a batch? Think of it as Priya's tilling-up sheet for the day. Every sale gets dropped into the open batch. End of day, batch closes; closed batch goes to settlement; settlement triggers payout. No batch open = no sales possible. (We'll see what happens when someone forgets in Chapter 5.)

Prasad packs up his tools and leaves. Priya is now LIVE.

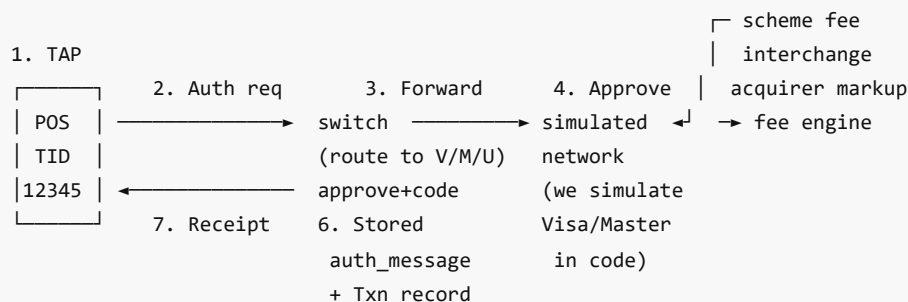
## Chapter 5 — Rahul buys a coffee (the heart of the system)

 Lunchtime. Rahul walks into Priya's Coffee Shop, orders a cappuccino + croissant, total ₹450.

He taps his Visa credit card on the terminal.

What happens in the next **2 seconds** is the whole reason this software exists.

### The full transaction flow



Step by step:

### 1. Tap

Card antenna ↔ terminal antenna. The terminal reads:

- BIN: 453201 (Visa, HDFC issued)
- Last 4: 0366
- Expiry, cardholder name, EMV chip data
- Contactless flag: yes

The terminal **never sends the full PAN over the wire** (PCI rule). It masks the middle: 453201\*\*\*\*\*0366 .

### 2. Auth request goes to AcquirerX

In the real world, the terminal dials AcquirerBank's switch over a leased line. In our simulation, when you click **+ New Transaction** in the UI, the frontend posts to:

```
POST /api/v1/transactions/authorize
{
  "terminalId": 1,
  "amount": 450.00,
  "currency": "INR",
  "panMasked": "453201*****0366",
  "txnType": "SALE"
}
```

At the api-gateway, this gets routed to **transaction-service:9093**.

### 3. Switch logic

SwitchService.authorize() does this checklist:

1. **Find the terminal** via Feign → terminal-service.getTerminalById(1) . Returns: TID 12345678 , status ACTIVE , merchant\_id 4 .
2. **Is there an open batch?** Query the local batch table for terminal 1, status OPEN. Yes — found one (Chapter 4.3). Good.
3. **Risk check** via Feign → risk-service.evaluate({pan, amount, merchant}) .
  - Is the PAN blacklisted? No.
  - Does any active rule fire? Rule "amount > 50000" doesn't apply (₹450).
  - Score: 5 / 100 — well below the threshold.
  - Decision: ALLOW .
4. **Fetch fee rules** for this MCC + network + region → returns: scheme 0.1%, interchange 1.5%, markup 0.4%.
5. **Compute fees:**
  - scheme = 0.10% of 450 = 0.45
  - interchange = 1.50% of 450 = 6.75
  - markup = 0.40% of 450 = 1.80
  - total fee = 9.00
  - net merchant amount = 450 - 9 = 441.00
6. **Send to network** — in our Phase-1 system, this is simulated. Approval is granted with code 00 and an auth code like 045123 . (In a real system, this is the moment AcquirerBank sends an ISO-8583 message to Visa, who routes it to HDFC, who checks Rahul's balance, and the answer flies back in ~700 ms.)
7. **Save:**

- `auth_message` row: `txn_type=SALE, amount=450, status=APPROVED, response_code=00, auth_code=045123, terminal_id=1, merchant_id=4, batch_id=1`.
- `txn` row: `amount=450, scheme_fee=0.45, interchange_fee=6.75, acquirer_markup=1.80, net_merchant_amount=441.00`.

#### 4-7. Response to terminal

Terminal beeps approval. Receipt prints with auth code `045123` . Rahul takes his coffee.

#### What just happened in money terms:

| Account                                                                                             | Movement                                                                       |
|-----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
|  Rahul's HDFC card | -₹450 (will appear on his statement)                                           |
|  Visa              | got a tiny scheme fee of ₹0.45                                                 |
|  HDFC (issuer)     | gets ₹6.75 interchange (since they bear the risk of Rahul not paying his bill) |
|  AcquirerBank      | keeps ₹1.80 markup as profit                                                   |
|  Priya             | will receive ₹441.00 in tomorrow's payout                                      |

Total fee Priya paid: ₹9 (= 2% of ₹450). That matches her MDR. ✓

## Chapter 6 — End of day: closing batches

🌙 11 PM. The store is closed. The terminal's auto-batch-close timer fires (we set it to hour 23 in the param profile).

In our system, Prasad logs in and clicks **Close Batch** on the terminal's Batch Control card. The batch row transitions:

```
status: OPEN → CLOSED
close_time: 2026-05-08 23:00:00
```

Throughout the day, Priya processed 50 transactions in this batch:

- 47 approved sales totalling ₹18,500
- 2 declined (insufficient funds, suspicion)
- 1 voided (cashier error: customer changed their mind)

Total approved:  $47 \times \text{average} \sim ₹400 = \text{₹18,500 gross}$  for the day.

## Chapter 7 — Settlement: Priya gets paid

🏠 The next morning (T+1, remember Priya's settlement cycle), Ravi (RECON officer) runs the settlement.

#### Manual run via UI

Sidebar → **Settlements** → **Run Settlement** → pick `Priya's Coffee Shop` .

***i** In a real system this happens automatically on a scheduler ( `SettlementScheduler` is in the codebase but disabled). Manual run is a backup mode for testing & recovery.*

```
SettlementService.settle(merchantId) runs:
```

1. **Find all approved, unsettled transactions** for merchant 4 since the last settlement → 47 transactions.

2. **Aggregate:**

- Gross = ₹18,500.00 (sum of amounts)
- Total fees = ₹370.00 (sum of all scheme + interchange + markup fees)
- Net = ₹18,130.00 (gross – fees)

3. **Apply reserve:** Priya's profile says 5% reserve.

- Reserve held = ₹906.50 (5% of ₹18,130)
- Payable today = ₹17,223.50

4. **Save records:**

- `settlement_batch` row: `merchant_id=4, period_start=yesterday, period_end=now, gross=18500, fees=370, net=18130, status=PAID`
- `payout` row: `bank_ref=HDFC0001234-00998877, amount=17223.50, status=POSTED`
- All 47 `txn` rows get marked `settled=true`

5. **Send notification** → Priya gets a "Settlement of ₹17,223.50 posted" notification (in-app for now).

 In the database after this:

```
settlement_batch: 1 row, status='PAID', net=18130
payout:           1 row, amount=17223.50, status='POSTED'
adjustment:       0 rows (none today)
notification:     new row for Priya
```

In a production system, the `payout` record would trigger an actual NEFT/RTGS transfer from AcquirerBank's nodal account to Priya's HDFC account. In Phase-1, we just record the intent; no actual money moves.

---

## Chapter 8 — Reconciliation: matching the books

 At end of day, multiple parties have records of what happened:

| Source                                | What they saw                                                           |
|---------------------------------------|-------------------------------------------------------------------------|
| <b>AcquirerX</b> (our switch)         | "We approved 47 transactions totalling ₹18,500"                         |
| <b>Visa network</b>                   | sends a daily file: "47 transactions cleared through us, total ₹18,500" |
| <b>AcquirerBank's banking partner</b> | sends a statement: "We received ₹18,500 from Visa overnight"            |

**Reconciliation** = checking these three lists agree.

In the perfect world they always match. In the real world they don't, because:

- Network might delay 1 transaction to the next day's file
- Bank might mistype an amount
- A timeout caused our system to record a transaction Visa didn't see (or vice versa)

### Loading the recon file

The next morning Ravi receives a CSV from the bank. He goes to **Reconciliation** → + **Load File**:

- Source: `BANK`
- File Date: `yesterday`
- Items: 47 rows with each transaction's reference + amount + bank's NEFT ref

The reconciliation engine runs:

- For each item in the bank file, find the matching `auth_message` in our DB by reference.
- If reference + amount match: `MATCHED` ✓
- If reference matches but amount differs: `MISMATCHED` (e.g. our 450 vs bank's 451 = 1-rupee error)
- If our `auth_message` exists but no bank file entry: `MissingTxn` exception
- If bank file has an entry we don't have: `Duplicate` or `Phantom` exception

For the perfect day, all 47 items show `MATCHED` . Match rate: 100%.

## When things don't match — exceptions

Suppose 1 transaction doesn't match — bank's amount is ₹501, ours is ₹500. That creates an `ExceptionCase` :

- Category: `AmountMismatch`
- Reference: `AUTH-104`
- Status: `OPEN`

Ravi opens the **Exceptions** tab on the Reconciliation page:

1. Sees the OPEN exception, clicks **Resolve**.
2. Investigates: contacts the bank's support → "yeah, sorry, our system added 1 rupee in error." Bank corrects.
3. In the modal, picks **Mark Resolved**, types notes (≥15 chars): `Bank confirmed 1-rupee adjustment error, corrected in next-day file` .
4. Clicks **Confirm**. Status flips to `RESOLVED` .

If the bank refuses to fix it and AcquirerBank decides to absorb the loss:

- Click **Write Off**, type notes (≥30 chars), confirm → `WRITTEN_OFF` .

---

## Chapter 9 — The risk team in action

 Meanwhile, Raj (RISK officer) is watching for fraud.

### Setting rules

Logged in as `raj` . **Risk** → **Rules** tab → + **New Rule**:

- Name: `Large amount review`
- Expression: `amount > 50000`
- Max Amount: `50000`
- Severity: `HIGH`
- Action: `REVIEW`

This rule says: "Any transaction over ₹50,000 must be reviewed before approval." (For a coffee shop this rarely fires; for an electronics store it would.)

### Adding a known-bad PAN to blacklist

Visa publishes a fraud bulletin: PAN starting with `499998` and ending in `9999` is stolen. Raj adds it to the blacklist:

**Risk** → **Blacklist** → + **New Entry**:

- Type: `PAN`
- Value: `499998*****9999`

- Reason: Visa fraud bulletin VFB-2026-0042 – confirmed compromise

From this moment, any auth attempt with that PAN is auto-declined. The risk-service's `evaluate()` checks the blacklist first.

### A risk event fires

Imagine someone tries to swipe `499998*****9999` at any merchant. The transaction goes through Switch → risk evaluation → blacklist hit → DECLINED. A `RiskEvent` row is created:

- `txn_id`: 105
- `rule`: BLACKLIST\_HIT
- `decision`: BLOCK
- `score`: 100

Raj sees this in the **Events** tab. He may decide to add the merchant to a watch list if they're seeing repeated attempts (someone is testing stolen cards at this terminal).

---

## Chapter 10 — A chargeback comes in

 Two weeks later. Rahul checks his HDFC credit card statement and sees the ₹450 from Priya's. He doesn't remember it. (He was at Priya's, but he forgot.) He clicks "Dispute" in his HDFC banking app.

### The dispute lifecycle

HDFC (the issuer) initiates a **chargeback** against AcquirerBank. In our system, this triggers (in real life — for testing we POST manually via Postman) the creation of a dispute case:

```
POST /api/v1/disputes
{
  "txnId": 67,
  "panMasked": "453201*****0366",
  "reasonCode": "FRAUD_4837"
}
```

Reason code `FRAUD_4837` is Visa's code for "Cardholder claims this is a fraudulent transaction."

A new `DisputeCase` row appears with:

- `stage`: RETRIEVAL
- `status`: OPEN
- `reason_code`: FRAUD\_4837
- `deadline`: 14 days from now

#### Stage 1: Retrieval

Visa, on behalf of HDFC, says "give me proof this transaction happened." Divya (DISPUTES officer) opens the dispute in **Disputes**, clicks the case, and works on Stage 1.

She reaches out to Priya: "Do you remember a ₹450 transaction on May 8 at lunch?" Priya digs out:

- The receipt with Rahul's signature
- A still from her CCTV at 12:43 showing Rahul at the counter
- The terminal log showing card insertion (EMV chip authenticated, not just stripe)

Divya uploads them as **DisputeDocuments**:

- Doc Type: RECEIPT , URI: https://files.acquirerbank.com/disputes/67/receipt.pdf
- Doc Type: DELIVERY\_PROOF , URI: https://files.acquirerbank.com/disputes/67/cctv-still.jpg

She also logs a **DisputeAction**:

- Action Type: SUBMIT\_EVIDENCE
- Notes: Receipt + CCTV still + EMV authenticated chip log submitted

Then she clicks **Advance Stage** → moves to **Chargeback**.

### Stage 2: Chargeback

Visa accepts Divya's evidence as adequate, and forwards it to HDFC. Now HDFC has 2 options:

- Accept Priya's evidence and drop the dispute
- Insist Rahul's claim is valid → goes to representment

Suppose HDFC says "the EMV chip log is good enough — Rahul authenticated with the chip, that's not fraud." HDFC withdraws the chargeback.

Divya clicks **Close Dispute**. Status: CLOSED . Priya keeps her ₹441.

### Stages 3 & 4: Representment, Arbitration

If HDFC pushes back ("Rahul says the chip was cloned"), Divya advances to **Representment** — gathering more evidence and replying. If neither side gives up, it escalates to **Arbitration**, where Visa makes the final binding decision. These are slow processes — weeks.

### The money flow during a dispute

While the dispute is open, AcquirerBank's books look like:

- ₹441 was paid to Priya on May 9
- ₹450 was clawed back from AcquirerBank by HDFC on May 22
- Net: AcquirerBank is out ₹9 from Priya's reserve, until resolution.

If Priya wins (representment accepted): money flows back to AcquirerBank.

If Priya loses: the ₹450 + chargeback fee comes out of her next settlement as an **Adjustment**. Ravi posts:

- Type: Chargeback
- Direction: Debit
- Amount: 450
- Reason: Chargeback FRAUD\_4837 lost on case #67 – not contested in time

Next time settlement runs, this adjustment is netted against new sales.

## Chapter 11 — Reports & dashboards

 At month end, Karthik (admin) wants to see how AcquirerBank did across all merchants. He opens **Reports**.

### Generated reports

He clicks + **New Report**:

- Scope: Network
- Period: last 30 days

The system queries all transactions, aggregates by network (V/M/U/LocalSim), and stores an `AcquirerReport` row:

```
metrics: {
  "Visa": { volume: 8200, value: 4_100_000, fees: 82_000, chargebackRate: 0.0023 },
  "Master": { volume: 5100, value: 2_600_000, fees: 52_000, chargebackRate: 0.0019 },
  "UPI": { volume: 3200, value: 800_000, fees: 8_000, chargebackRate: 0.0001 }
}
```

Chargeback rate (number of chargebacks / number of sales) is the bank's North Star metric. Visa charges *the merchant and the acquirer* extra fees if their chargeback rate goes above 1%. Below 0.5% is excellent.

Priya's chargeback rate is 1/47 from the May 8 case — but it was contested and won. So her "chargeback rate" stays at 0% if we count by lost-disputes. (Different metrics measure different things.)

## Chapter 12 — Notifications

-  Throughout the story, every meaningful event generated an in-app notification:
- May 8, 9 PM — *"Batch closed for terminal 12345678 — 47 approved txns"* (POS\_OPS, RECON)
  - May 9, 10 AM — *"Settlement posted: Priya's Coffee Shop, ₹17,223.50"* (RECON, MERCHANT\_OPS, ADMIN)
  - May 22, 11 AM — *"Dispute opened on txn 67"* (DISPUTES)
  - May 22, 11 AM — *"Risk event fired: rule BLACKLIST\_HIT"* (RISK)
  - May 28, 4 PM — *"Recon file processed: 47/47 matched"* (RECON)
  - June 5, 9 AM — *"Dispute closed: case 67 — won"* (DISPUTES)

Each user sees only their own categories. Click the bell icon, mark as read, or dismiss.

## Chapter 13 — A day in the life — summary

| Time          | Who      | Action                                           | What changes                                                                                |
|---------------|----------|--------------------------------------------------|---------------------------------------------------------------------------------------------|
| 09:00         | Prasad   | Opens batch on TID 12345678                      | <code>batch.status=OPEN</code>                                                              |
| 12:43         | Rahul    | Buys coffee for ₹450                             | <code>new auth_message + txn rows</code>                                                    |
| 12:43         | (system) | Risk evaluates, fees computed, network simulated | <code>risk_event</code> , <code>fee_rule</code> consulted                                   |
| 23:00         | (system) | Terminal auto-closes batch                       | <code>batch.status=CLOSED</code>                                                            |
| Next 09:00    | Ravi     | Runs settlement for Priya                        | <code>settlement_batch</code> , payout, <code>txn.settled=true</code> , notification        |
| Next 14:00    | Bank     | Sends recon file                                 | (next day)                                                                                  |
| Next +1 11:00 | Ravi     | Loads + processes recon                          | <code>recon_file</code> , <code>recon_item.match_status=MATCHED</code>                      |
| 2 weeks later | HDFC     | Files chargeback                                 | <code>dispute_case.stage=RETRIEVAL</code>                                                   |
| 2 weeks + 2   | Divya    | Submits evidence, advances stage                 | <code>dispute_document</code> , <code>dispute_action</code> , <code>stage→CHARGEBACK</code> |



|               |          |                                  |                            |
|---------------|----------|----------------------------------|----------------------------|
| 3 weeks later | (system) | Dispute closed in Priya's favour | dispute_case.status=CLOSED |
|---------------|----------|----------------------------------|----------------------------|

That's a complete merchant lifecycle — onboarding, daily ops, settlements, exceptions, disputes — implemented across your 9 microservices and 1 frontend. Every business event in this story corresponds to actual code in your repo.

## Where each story chapter lives in the code

If you want to read the actual implementation:

| Story                                                      | Backend service     | Key class                                                | Frontend page                        |
|------------------------------------------------------------|---------------------|----------------------------------------------------------|--------------------------------------|
| Login & user mgmt                                          | auth-service        | AuthService.java                                         | /login, /profile, /admin/users       |
| Onboarding (merchant, KYC, pricing, settle profile, store) | merchant-service    | MerchantService, MerchantOnboardingService, StoreService | /merchants, /stores                  |
| Provisioning terminal & param profile                      | terminal-service    | TerminalService, ParamProfileService                     | /terminals                           |
| Authorization (Rahul's tap)                                | transaction-service | SwitchService.authorize()                                | /transactions/new, /transactions/:id |
| Fee computation                                            | transaction-service | FeeService, FeeRuleMatcher                               | /admin/fee-rules                     |
| Risk evaluation                                            | risk-service        | RiskService.evaluate()                                   | /risk                                |
| Batch open/close                                           | transaction-service | SwitchService.openBatch(), closeBatch()                  | terminal detail → Batch Control      |
| Settlement & payout                                        | settlement-service  | SettlementService.settle()                               | /settlement                          |
| Adjustments                                                | settlement-service  | AdjustmentService                                        | /settlement → New Adjustment         |
| Reconciliation                                             | ops-service         | ReconciliationService                                    | /reconciliation                      |
| Disputes                                                   | ops-service         | DisputeService                                           | /disputes                            |
| Reports                                                    | ops-service         | ReportService                                            | /reports                             |
| Notifications                                              | ops-service         | NotificationService                                      | bell icon, /notifications            |

## Why microservices? (the question every newbie asks)

Could all this fit in one Java program? **Yes.** The `backend/` folder in your repo proves it — that's a monolith version with everything stuffed together.

So why split into 9 services?

1. **Different teams own different parts.** The risk team can deploy fraud rules without redeploying merchant onboarding.
2. **Scale independently.** During Diwali sales, transaction-service handles 1000 txns/sec while merchant-service is barely used. With microservices you scale only what's hot.
3. **Tech freedom.** If risk needs Python ML libraries, only that service goes Python; the others stay Java.
4. **Fault isolation.** If reports crash, transactions still authorize. (The chargeback dashboard is down — but Priya's customer can still pay.)

The cost is complexity: Eureka, Feign, JWT shared between services, distributed logs, etc.

For this project, microservices are mostly a learning exercise. In a real bank, the trade-off becomes worth it once the team grows beyond ~20 engineers.

---

## You're not expected to remember everything

You don't need to memorize this guide. You need to *recognize* the words when you see them in the code or the playbook. Most people on a payment team learn this gradually over months.

The mental model that matters:

```
Sign up the shop → install the device → customer taps →  
  approve in 2 sec → end of day batch → settle to bank →  
    reconcile with the bank's file → ...if customer disputes →  
      walk it through the 4 stages → close it
```

That's payment processing. Everything in your codebase serves one of these steps.

Good luck. 🚀